

AI ASSISTED CODING

ASSIGNMENT -9.1

Name: Geethanjali

H.T.No: 2303A51005

Problem 1:

Consider the following Python function:

```
def find_max(numbers): return  
max(numbers)
```

Task:

- Write documentation for the function in all three formats:

(a) Docstring

(b) Inline comments

(c) Google-style documentation

- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

```
1  def find_max(numbers):  
2      """  
3          This function takes a list of numbers as input and returns the maximum number in the list.  
4          Parameters:  
5          numbers (list): A list of numerical values.  
6          Returns:  
7          The maximum number in the list. If the list is empty, it returns None.  
8          """  
9      if not numbers:  
10         return None # Return None for an empty list  
11  
12         max_number = numbers[0] # Initialize max_number to the first element  
13  
14         for num in numbers:  
15             if num > max_number:  
16                 max_number = num # Update max_number if a larger number is found  
17  
18         return max_number  
19 # Example usage  
20 numbers = [3, 5, 7, 2, 8]  
21 max_value = find_max(numbers) # Storing the maximum value in a variable  
22 print(f"The maximum number in the list is: {max_value}") # Printing the maximum value found in the list  
23 print(find_max.__doc__)  
24
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
<pre>Returns: The maximum number in the list. If the list is empty, it returns None. The maximum number in the list is: 8 The maximum number in the list is: 8 PS C:\Users\gudah\OneDrive\Documents\AIAC></pre>				

```

26 def find_max(numbers):
27     if not numbers:
28         return None # Return None for an empty list
29
30     max_number = numbers[0] # Initialize max_number to the first element
31
32     for num in numbers:
33         if num > max_number:
34             max_number = num # Update max_number if a larger number is found
35
36     return max_number
37
38 # Example usage
39 numbers = [3, 5, 7, 2, 8]
40 max_value = find_max(numbers) # storing the maximum value in a variable
41 print(f"The maximum number in the list is: {max_value}") # printing the maximum value found in the list

```

```

43 #Generate python code for returning the maximum value from a list of numbers and print the result and give google style docstrings for the function
44 def find_max(numbers:list)->int:
45     """
46     Returns the maximum value from a list of numbers.
47
48     Args:
49         numbers (list): A list of numerical values.
50     Returns:
51         int: The maximum number in the list. If the list is empty, it returns None.
52     """
53     if not numbers:
54         return None # Return None for an empty list
55
56     max_number = numbers[0] # Initialize max_number to the first element
57
58     for num in numbers:
59         if num > max_number:
60             max_number = num # Update max_number if a larger number is found
61
62     return max_number
63
64 # Example usage
65 numbers = [3, 5, 7, 2, 8]
66 max_value = find_max(numbers) # Storing the maximum value in a variable
67 print(f"The maximum number in the list is: {max_value}") # Printing the maximum value found in the list

```

Docstring: Written inside triple quotes below the function, it explains what the function does, its inputs and outputs, and can be viewed using `help()`, making it simple and easy to write.

Inline Comments: Written using `#`, they explain the logic step by step inside the code and help developers understand how each part works clearly.

Google-Style Documentation: A structured form of docstring that uses sections like `Args` and `Returns`, looks professional, and is suitable for large projects.

Best Style: Google-style documentation is best for a mathematical utilities library because it clearly explains inputs, outputs, and formulas. It keeps documentation organized and easy to understand for other developers.

Problem 2: Consider the following Python function: `def login(user, password, credentials):`

`return credentials.get(user) == password` Task:

1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

```

70 def login(user,password,credentials):
71     """
72     This function checks if the provided username and password match any entry in the credentials dictionary.
73
74     Args:
75     |   user (str): The username to be checked.
76     |   password (str): The password to be checked.
77     |   credentials (dict): A dictionary where keys are usernames and values are passwords.
78
79     Returns:
80     |   bool: True if the username and password match an entry in the credentials, False otherwise.
81     """
82     if user in credentials and credentials[user] == password:
83     |   return True
84     |   else:
85     |       return False
86
87 # Example usage
88 credentials = {"admin": "admin123", "user1": "password1", "user2": "password2"}
89 print(login("admin", "admin123", credentials)) # Output: True
90 print(login("user1", "wrongpassword", credentials)) # Output: False
91 print(login.__doc__)

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\gudah\OneDrive\Documents\AIAC> & C:\Python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_9.1.py
True
False

```

This function checks if the provided username and password match any entry in the credentials dictionary.

Args:

user (str): The username to be checked.
password (str): The password to be checked.
credentials (dict): A dictionary where keys are usernames and values are passwords.

Returns:

bool: True if the username and password match an entry in the credentials, False otherwise.

```

94 def login(user,password,credentials):
95     if user in credentials and credentials[user] == password:#checking if the user exists in the credentials and if the password matches
96     |   return True#if the user exists and the password matches, return True
97     |   else:#if the user does not exist or the password does not match, return False
98     |       return False
99
100 # Example usage
101 credentials = {"admin": "admin123", "user1": "password1", "user2": "password2"}#a dictionary containing usernames and their corresponding passwords
102 print(login("admin", "admin123", credentials)) # Output: True
103 print(login("user1", "wrongpassword", credentials)) # Output: False
104
105 #Generate python code for a login function that checks if the provided username and password match any entry in a credentials dictionary and print the result and give goo
106 def login(user:str,password:str,credentials:dict)->bool:
107     """
108     This function checks if the provided username and password match any entry in the credentials dictionary.
109
110     Args:
111     |   user (str): The username to be checked.
112     |   password (str): The password to be checked.
113     |   credentials (dict): A dictionary where keys are usernames and values are passwords.
114
115     Returns:

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

PS C:\Users\gudah\OneDrive\Documents\AIAC> & C:\Python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_9.1.py

```

Returns:

bool: True if the username and password match an entry in the credentials, False otherwise.

True

False

True

False

```

PS C:\Users\gudah\OneDrive\Documents\AIAC>

```

```

105 #Generate python code for a login function that checks if the provided username and password match any entry in a credentials dictionary and print the result and give
106 def login(user:str,password:str,credentials:dict)->bool:
107     """
108     This function checks if the provided username and password match any entry in the credentials dictionary.
109
110     Args:
111     |   user (str): The username to be checked.
112     |   password (str): The password to be checked.
113     |   credentials (dict): A dictionary where keys are usernames and values are passwords.
114
115     Returns:
116     |   bool: True if the username and password match an entry in the credentials, False otherwise.
117     """
118     if user in credentials and credentials[user] == password:
119     |   return True
120     |   else:
121     |       return False
122
123 # Example usage
124 credentials = {"admin": "admin123", "user1": "password1", "user2": "password2"}#a dictionary containing usernames and their corresponding passwords
125 print(login("admin", "admin123", credentials)) # Output: True
126 print(login("user1", "wrongpassword", credentials)) # Output: False

```

Google-style documentation is most helpful for new developers. It clearly explains inputs, outputs, and purpose in a structured way. New developers can quickly understand how to use the function. It keeps documentation neat and professional. It is best for onboarding in real projects.

Problem 3: Calculator (Automatic Documentation Generation)

Task: Design a Python module named `calculator.py` and demonstrate automatic documentation generation.

Instructions:

1. Create a Python module `calculator.py` that includes the following functions, each written with appropriate docstrings:

o `add(a, b)` – returns the sum of two numbers o

`subtract(a, b)` – returns the difference of two numbers o

`multiply(a, b)` – returns the product of two numbers o

`divide(a, b)` – returns the quotient of two numbers

2. Display the module documentation in the terminal using Python's documentation tools.

3. Generate and export the module documentation in HTML format using the `pydoc` utility, and open the generated HTML file in a web browser to verify the output.

```
calculator.py > ...
1
2 This module provides basic arithmetic operations: addition, subtraction, multiplication, and division.
3 Each function takes two numbers as input and returns the result of the respective operation.
4 The functions also include Google-style docstrings that describe their purpose, arguments, and return values.
5 The module can be used as a simple calculator for performing basic mathematical operations.
6 Example usage of each function is provided,
7 along with the ability to print the documentation for each function using the __doc__ attribute.
8
9 """
10
11 def add(a,b):
12     """
13     This function takes two numbers as input and returns their sum.
14
15     Args:
16     | a (int): The first number.
17     | b (int): The second number.
18     Returns:
19     | int: The sum of the two numbers.
20     """
21     return a + b
22 print("The sum of 5 and 3 is:", add(5, 3)) # Example usage of the add function and printing the result
23 print(add.__doc__)
24
25 def subtract(a,b):
26     """
27     This function takes two numbers as input and returns their difference.
28
29     Args:
30     | a (int): The first number.
31     | b (int): The second number.
32     Returns:
33     | int: The difference of the two numbers.
34     """
35     return a - b
36 print("The difference between 5 and 3 is:", subtract(5, 3)) # Example usage of the subtract function and printing the result
37 print(subtract.__doc__)
```

```

39 def multiply(a,b):
40     """
41     This function takes two numbers as input and returns their product.
42
43     Args:
44         a (int): The first number.
45         b (int): The second number.
46     Returns:
47         int: The product of the two numbers.
48     """
49     return a * b
50 print("The product of 5 and 3 is:", multiply(5, 3)) # Example usage of the multiply function and printing the result
51 print(multiply.__doc__)
52
53 def divide(a,b):
54     """
55     This function takes two numbers as input and returns their quotient.
56
57     Args:
58         a (int): The first number (dividend).
59         b (int): The second number (divisor).
60     Returns:
61         float: The quotient of the two numbers. If the divisor is zero, it returns None.
62     """
63     if b == 0:
64         return None # Return None for division by zero
65     return a / b
66 print("The quotient of 5 and 3 is:", divide(5, 3)) # Example usage of the divide function and printing the result
67 print("The quotient of 5 and 0 is:", divide(5, 0))
68
69 import calculator
70 print(calculator.add.__doc__) # Printing the documentation for the add function
71

```

PS C:\Users\gudah\OneDrive\Documents\AIAC> python -m pydoc calculator

This function takes two numbers as input and returns their sum.

Args:

a (int): The first number.
b (int): The second number.

Returns:

int: The sum of the two numbers.

Help on module calculator:

NAME

calculator

DESCRIPTION

This module provides basic arithmetic operations: addition, subtraction, multiplication, and division. Each function takes two numbers as input and returns the result of the respective operation. The functions also include Google-style docstrings that describe their purpose, arguments, and return values. The module can be used as a simple calculator for performing basic mathematical operations. Example usage of each function is provided, along with the ability to print the documentation for each function using the `__doc__` attribute.

FUNCTIONS

`add(a, b)`

This function takes two numbers as input and returns their sum.

Args:

a (int): The first number.
b (int): The second number.

Returns:

int: The sum of the two numbers.

`divide(a, b)`

This function takes two numbers as input and returns their quotient.

Args:

a (int): The first number (dividend).
b (int): The second number (divisor).

Returns:

calculator <c:\users\gudah\onedrive\documents\aiac\calculator.py>

This module provides basic arithmetic operations: addition, subtraction, multiplication, and division. Each function takes two numbers as input and returns the result of the respective operation. The functions also include Google-style docstrings that describe their purpose, arguments, and return values. The module can be used as a simple calculator for performing basic mathematical operations. Example usage of each function is provided, along with the ability to print the documentation for each function using the `__doc__` attribute.

Modules

[calculator](#)

Functions

add(a, b)

This function takes two numbers as input and returns their sum.

Args:

a (int): The first number.

b (int): The second number.

Returns:

int: The sum of the two numbers.

divide(a, b)

This function takes two numbers as input and returns their quotient.

Args:

a (int): The first number (dividend).

b (int): The second number (divisor).

Returns:

float: The quotient of the two numbers. If the divisor is zero, it returns None.

multiply(a, b)

This function takes two numbers as input and returns their product.

Problem 4: Conversion Utilities Module

Task:

1. Write a module named `conversion.py` with functions:

o `decimal_to_binary(n)` o

`binary_to_decimal(b)` o

`decimal_to_hexadecimal(n)`

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

```

1  """
2  This module provides functions for converting between different number systems: decimal to binary, binary to decimal, and decimal to hexadecimal.
3  Each function includes a Google-style docstring that describes its purpose, arguments, and return value.
4  The module can be used to perform various conversions between these number systems,
5  and example usage of each function is provided, along with the ability to print the
6  documentation for each function using the __doc__ attribute.
7  """
8
9
10 def decimal_to_binaary(n):
11     """
12     This function converts a decimal number to its binary representation.
13
14     Args:
15     |   n (int): The decimal number to be converted.
16
17     Returns:
18     |   str: The binary representation of the input decimal number.
19     """
20     if n == 0:
21         return "0"
22
23     binary = ""
24     while n > 0:
25         binary = str(n % 2) + binary
26         n //= 2
27
28     return binary
29 print("The binary representation of 10 is:", decimal_to_binaary(10)) # Example usage of the decimal_to_binaary function and printing the result
30 print(decimal_to_binaary.__doc__)
31

```

```

32 def binary_to_decimal(b):
33     """
34     This function converts a binary number (given as a string) to its decimal representation.
35
36     Args:
37     |   b (str): The binary number to be converted, represented as a string of '0's and '1's.
38
39     Returns:
40     |   int: The decimal representation of the input binary number.
41     """
42     decimal = 0
43     power = 0
44
45     for digit in reversed(b):
46         if digit == '1':
47             decimal += 2 ** power
48             power += 1
49
50     return decimal
51 print("The decimal representation of '1010' is:", binary_to_decimal('1010')) # Example usage of the binary_to_decimal function and printing the result
52 print(binary_to_decimal.__doc__)

```

```

54 def decimal_to_hexadecimal(n):
55     """
56     This function converts a decimal number to its hexadecimal representation.
57
58     Args:
59     |   n (int): The decimal number to be converted.
60
61     Returns:
62     |   str: The hexadecimal representation of the input decimal number.
63     """
64     if n == 0:
65         return "0"
66
67     hexadecimal = ""
68     hex_digits = "0123456789ABCDEF"
69
70     while n > 0:
71         hexadecimal = hex_digits[n % 16] + hexadecimal
72         n //= 16
73
74     return hexadecimal
75 print("The hexadecimal representation of 255 is:", decimal_to_hexadecimal(255)) # Example usage of the decimal_to_hexadecimal function and printing the result
76 print(decimal_to_hexadecimal.__doc__)
77
78 import conversions
79 print(conversions.__doc__)

```

```
PS C:\Users\gudah\OneDrive\Documents\AIAC> & C:\Python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/conversions.py
• The binary representation of 10 is: 1010
```

This function converts a decimal number to its binary representation.

Args:
n (int): The decimal number to be converted.

Returns:
str: The binary representation of the input decimal number.

The decimal representation of '1010' is: 10

This function converts a binary number (given as a string) to its decimal representation.

Args:
b (str): The binary number to be converted, represented as a string of '0's and '1's.

Returns:
int: The decimal representation of the input binary number.

The hexadecimal representation of 255 is: FF

This function converts a decimal number to its hexadecimal representation.

Args:
n (int): The decimal number to be converted.

Returns:
str: The hexadecimal representation of the input decimal number.

The binary representation of 10 is: 1010

This function converts a decimal number to its binary representation.

Args:
n (int): The decimal number to be converted.

Returns:
str: The binary representation of the input decimal number.

The decimal representation of '1010' is: 10

This function converts a binary number (given as a string) to its decimal representation.

Args:
b (str): The binary number to be converted, represented as a string of '0's and '1's.

Returns:
int: The decimal representation of the input binary number.

The hexadecimal representation of 255 is: FF

This function converts a decimal number to its hexadecimal representation.

Args:
n (int): The decimal number to be converted.

Returns:
str: The hexadecimal representation of the input decimal number.

This module provides functions for converting between different number systems: decimal to binary, binary to decimal, and decimal to hexadecimal. Each function includes a Google-style docstring that describes its purpose, arguments, and return value. The module can be used to perform various conversions between these number systems, and example usage of each function is provided, along with the ability to print the documentation for each function using the `__doc__` attribute.

This module provides functions for converting between different number systems: decimal to binary, binary to decimal, and decimal to hexadecimal. Each function includes a Google-style docstring that describes its purpose, arguments, and return value. The module can be used to perform various conversions between these number systems, and example usage of each function is provided, along with the ability to print the documentation for each function using the `__doc__` attribute.

[index](#)

conversions [c:\users\gudah\onedrive\documents\aiac\conversions.py](#)

This module provides functions for converting between different number systems: decimal to binary, binary to decimal, and decimal to hexadecimal. Each function includes a Google-style docstring that describes its purpose, arguments, and return value. The module can be used to perform various conversions between these number systems, and example usage of each function is provided, along with the ability to print the documentation for each function using the `__doc__` attribute.

Modules

[conversions](#)

Functions

binary_to_decimal(b)

This function converts a binary number (given as a string) to its decimal representation.

Args:

b (str): The binary number to be converted, represented as a string of '0's and '1's.

Returns:

int: The decimal representation of the input binary number.

decimal_to_binaary(n)

This function converts a decimal number to its binary representation.

Args:

n (int): The decimal number to be converted.

Returns:

str: The binary representation of the input decimal number.

decimal_to_hexadecimal(n)

This function converts a decimal number to its hexadecimal representation.

Problem 5 – Course Management Module Task:

1. Create a module `course.py` with functions:

o `add_course(course_id, name, credits)`

o `remove_course(course_id)` o

`get_course(course_id)`

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

```
1  """
2  This module defines functions for managing courses in a course catalog.
3  It includes functions to add a course, remove a course, and retrieve course information.
4  | Each function is documented with Google style docstrings to explain its purpose, parameters, and return values.
5  The module can be used to simulate a simple course management system, allowing users to add, remove, and retrieve course information.
6  | Example usage of each function is provided, along with the ability to print the documentation for each function using the __doc__ attribute.
7  """
8  def add_course(course_id,name,credits):
9      """
10     This function takes a course ID, name, and credits as input and returns a dictionary representing the course.
11     Args:
12     |   course_id (str): The unique identifier for the course.
13     |   name (str): The name of the course.
14     |   credits (int): The number of credits for the course.
15     Returns:
16     |   dict: A dictionary containing the course information with keys 'course_id', 'name', and 'credits'.
17     """
18     return {
19         "course_id": course_id,
20         "name": name,
21         "credits": credits
22     }
23 # Example usage of the add_course function and printing the result
24 course_info = add_course("CS101", "Introduction to Computer Science", 4)
25 print(course_info)
26 print(add_course.__doc__)
27
```

```

29 def remove_course(course_id):
30     """
31     This function takes a course ID as input and simulates the removal of a course from a course catalog.
32     Args:course_id (str): The unique identifier for the course to be removed.
33     Returns:
34     str: A message indicating that the course has been removed.
35     """
36     return f"Course with ID {course_id} has been removed from the catalog."
37 # Example usage of the remove_course function and printing the result
38 print(remove_course("CS101"))
39 print(remove_course.__doc__)
40
41 def get_course(course_id):
42     """
43     This function takes a course ID as input and simulates retrieving course information from a course catalog.
44     Args:course_id (str): The unique identifier for the course to be retrieved.
45     Returns:
46     dict: A dictionary containing the course information if found, or a message indicating that the course was not found.
47     """
48     # Simulating a course catalog with a dictionary
49     course_catalog = {
50         "CS101": {"name": "Introduction to Computer Science", "credits": 4},
51         "MATH201": {"name": "Calculus I", "credits": 3},
52         "ENG301": {"name": "English Literature", "credits": 2}
53     }
54     if course_id in course_catalog:
55         return course_catalog[course_id]
56     else:
57         return f"Course with ID {course_id} not found in the catalog."
58 # Example usage of the get_course function and printing the result
59 print(get_course("CS101"))
60 print(get_course("HIST101"))
61 print(get_course.__doc__)
62
63 import course
64 print(course.__doc__)

```

● PS C:\Users\gudah\OneDrive\Documents\AIAC> & C:\Python314\python.exe c:/Users/gudah/OneDrive/Documents/AIAC/course.py
{'course_id': 'CS101', 'name': 'Introduction to Computer Science', 'credits': 4}

This function takes a course ID, name, and credits as input and returns a dictionary representing the course.

Args:

course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:

dict: A dictionary containing the course information with keys 'course_id', 'name', and 'credits'.

Course with ID CS101 has been removed from the catalog.

This function takes a course ID as input and simulates the removal of a course from a course catalog.

Args:course_id (str): The unique identifier for the course to be removed.

Returns:

str: A message indicating that the course has been removed.

{'name': 'Introduction to Computer Science', 'credits': 4}

Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.

Args:course_id (str): The unique identifier for the course to be retrieved.

Returns:

dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

{'course_id': 'CS101', 'name': 'Introduction to Computer Science', 'credits': 4}

This function takes a course ID, name, and credits as input and returns a dictionary representing the course.

Args:

course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:

dict: A dictionary containing the course information with keys 'course_id', 'name', and 'credits'.

Course with ID CS101 has been removed from the catalog.

```
This function takes a course ID as input and simulates the removal of a course from a course catalog.
Args:course_id (str): The unique identifier for the course to be removed.
Returns:
str: A message indicating that the course has been removed.

{'name': 'Introduction to Computer Science', 'credits': 4}
Course with ID HIST101 not found in the catalog.

This function takes a course ID as input and simulates retrieving course information from a course catalog.
Args:course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

This module defines functions for managing courses in a course catalog.
It includes functions to add a course, remove a course, and retrieve course information.
Each function is documented with Google style docstrings to explain its purpose, parameters, and return values.
The module can be used to simulate a simple course management system, allowing users to add, remove, and retrieve course information.
Example usage of each function is provided, along with the ability to print the documentation for each function using the __doc__ attribute.
```

[index](#)

course [c:\users\gudah\onedrive\documents\aiac\course.py](#)

This module defines functions for managing courses in a course catalog.
It includes functions to add a course, remove a course, and retrieve course information.
Each function is documented with Google style docstrings to explain its purpose, parameters, and return values.
The module can be used to simulate a simple course management system, allowing users to add, remove, and retrieve course information.
Example usage of each function is provided, along with the ability to print the documentation for each function using the `__doc__` attribute.

Modules

[course](#)

Functions

add_course(course_id, name, credits)

This function takes a course ID, name, and credits as input and returns a dictionary representing the course.

Args:

course_id (str): The unique identifier for the course.

name (str): The name of the course.

credits (int): The number of credits for the course.

Returns:

dict: A dictionary containing the course information with keys 'course_id', 'name', and 'credits'.

get_course(course_id)

This function takes a course ID as input and simulates retrieving course information from a course catalog.

Args:course_id (str): The unique identifier for the course to be retrieved.

Returns:

dict: A dictionary containing the course information if found, or a message indicating that the course was not found.

remove_course(course_id)

This function takes a course ID as input and simulates the removal of a course from a course catalog.

Args:course_id (str): The unique identifier for the course to be removed.

Returns:

str: A message indicating that the course has been removed.